From Chest X-rays to Medical Reports: A Deep Learning Approach

Frandina Ilaria 264232 Siciliano Samuele 263633 Spadafora Pierpaolo 263722

1 Introduction

In this report, we present a comprehensive multi-task deep learning approach for chest X-ray analysis. Our goal is to develop a unified model that simultaneously performs disease classification, radiological report generation, and image reconstruction.

The model leverages state-of-the-art architectures by combining ResNet-based encoders for extracting image features, GPT-2 for generating textual reports, and a Variational Autoencoder (VAE) for reconstructing input images. This integration improves the interpretability of the diagnostic process while ensuring that the learned representations remain robust and clinically relevant.

To achieve this, we first implemented a rigorous data cleaning pipeline to ensure high-quality inputs by filtering out incomplete or low-quality reports and images. The resulting dataset includes both frontal and lateral X-ray views, each accompanied by detailed clinical findings.

Our multi-task model is trained end-to-end with a composite loss function that integrates classification loss (augmented with focal loss to address class imbalance), caption generation loss (using teacher forcing techniques), and VAE reconstruction loss (MSE and KLD). We evaluate the model using classification accuracy, BLEU, and ROUGE scores across different tasks.

2 Dataset Description

The initial dataset consisted of:

- A collection of frontal and lateral X-ray images.
- Two CSV files containing, respectively, the projection information of the images and the associated reports (including clinical findings and pathologies).

Item	**Value**
Total number of UIDs	2728
Number of Validated Reports	5574
Number of Images Used	5574 (2728 per view)

Table 1: Initial Dataset Statistics

3 Data Preprocessing and Cleaning

To ensure data quality and reduce potential noise during training, we performed the following steps:

1. **Elimination of Incomplete Records:**
   - All records with one or more essential fields (e.g., `findings`, `Problems`, `MeSH`) that were empty or contained only blank strings were removed.
2. **Verification of Image Consistency:**
   - Records that did not contain exactly one frontal and one lateral X-ray were excluded to guarantee the presence of both views for each UID.
3. **Filtering Based on Pathology Frequency:**
   - Pathologies whose total occurrences did not exceed a threshold (in this case, 20) were discarded.
4. **Grouping into Macro-categories:**
   - Pathologies were grouped into macro-categories. The categorization was adapted based on the type of classification:
     - **Binary:** pathology vs. no pathology.
     - **Ternary:** pulmonary problems versus other issues.
     - **Multiclass:** division into 5 (or more) categories.

It should be noted that due to the intrinsic nature of the dataset (X-rays), conventional data augmentation techniques could not be applied effectively. For instance, a simple horizontal flip would reverse the positions of the organs, potentially introducing errors and misleading conclusions.

4 Dataset Splitting and DataLoaders

After cleaning, the dataset was split as follows:

- **Training set:** 60%
- **Validation set:** 20%
- **Test set:** 20%

To simplify loading data onto the GPU and to concatenate the images (frontal and lateral) with the textual reports, a custom `Dataset` class (based on `torch.utils.data.Dataset`) was implemented. Corresponding DataLoaders were then created for training, validation, and testing.

5 Model Architecture

The proposed model, *AllegroChirurgoModel*, integrates several components:

5.1 Image Encoders

- A pre-trained **ResNet50** is used for each view (frontal and lateral).
- The final layer is replaced with an identity layer (`nn.Identity`) to extract features.
- Most of the ResNet50 layers are frozen, leaving only the last 6 layers trainable to allow fine-tuning during training.

5.2 Classification Branch

The features extracted from the two ResNets are:

- Concatenated and passed through several fully connected layers (with dropout and layer normalization) to obtain the final classification logits.
- The classification loss is computed using **Cross-Entropy Loss** (and, where necessary, **Focal Loss** to address underrepresented classes).

5.3 Report Generation Decoder

- The combined features are projected into a space with a dimension compatible with the decoder input (GPT-2 uses embeddings of dimension 768, while the feature vector extracted from ResNet is 2048).
- The decoder is based on **GPT-2 LMHeadModel**, which is adapted by resizing the vocabulary based on the tokenizer used.
- Teacher forcing is employed during training to improve the quality of report generation.

5.4 VAE Branch for Image Reconstruction

- From the combined feature vector, the parameters μ and $\log \sigma^2$ are extracted for the **reparameterization trick**.
- A fully connected layer projects the latent vector into a dimension that is then "deconvolved" using a series of **ConvTranspose2d** layers to reconstruct both the frontal and lateral images.
- The VAE loss is calculated as the sum of the **Mean Squared Error (MSE)** for reconstruction and the **Kullback-Leibler Divergence (KLD)**.

6 Training and Evaluation Process

In our initial approach, the following components were optimized simultaneously during training:

- **Classification Loss:** Computed on the logits from the classification branch.
- **Report Generation Loss:** Using the GPT-2 decoder, the loss is computed by comparing the generated sequences with the ground truth.

- **VAE Loss:** A combination of reconstruction loss and KLD.

The total loss function is a linear combination of these components, weighted by the coefficients `ALPHA_MAIN_MODEL` (for the caption loss) and `BETA_VAE` (for the VAE loss). For training, we employed:

- A **Scheduler** to reduce the learning rate upon plateau.
- **Early Stopping** to halt training when no improvements in validation loss are observed.

Evaluation metrics include:

- **Accuracy** for classification.
- **BLEU** and **ROUGE-L** scores to assess the quality of the generated reports.

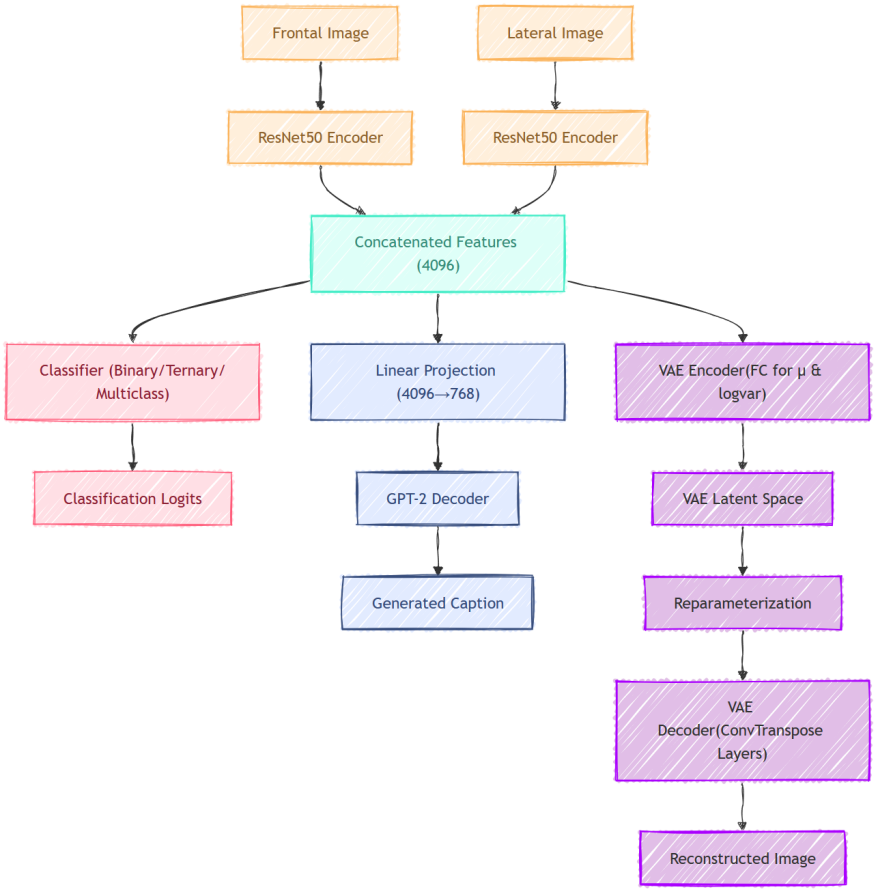

Figure 1: Overall training and evaluation pipeline.

This approach enabled us to develop a model capable of providing both diagnosis and report generation from the same latent space.
However, it became evident that combining multiple tasks into a single model, especially with an insufficient amount of data, led to less than satisfactory results.

We implemented Focal Loss to mitigate the issue of underrepresented classes in the classification task and adjusted the weighting of the losses, reducing the impact of both

the text prediction and the VAE losses.

For the training process, we implemented two key optimization strategies:

- A **Learning Rate Scheduler (ReduceLROnPlateau)** that monitors the validation loss and reduces the learning rate by a factor of 0.5 if no improvement is observed for 2 consecutive epochs. This helps the model converge to better local minima when the training process starts to plateau.
- An **Early Stopping** mechanism that tracks the validation loss and stops the training if no improvement is detected for 3 consecutive epochs. This, along with dropout in the classifier network, prevents overfitting and ensures optimal model performance while saving computational resources.

Despite these measures, we observed that the model's accuracy in the binary classification scenario fell below 70%, with significant challenges in generating accurate text and faithfully reconstructing images.

True Caption:
The cardiac contours are normal. The lungs are clear. Thoracic spondylosis.

Pred Caption (Model Output):
The heart and lungs have XXXX XXXX in the interval. Both lungs are clear and expanded. No pleural air collections. Heart and mediastinum normal.

	Class 1	**Class 2**	**Class 3**	**Class 4**	**Class 5**	**Total Accuracy**
Binary	61.46%	68.33%	-	-	-	65.75%
Ternary	15.12%	68.20%	39.52%	-	-	41.76%
Multiclass	38.54%	8.29%	14.71%	41.30%	34.09%	24.91%

Table 2: Classification and Text Generation Results

	BLEU-1	**BLEU-2**	**BLEU-3**	**BLEU-4**	**ROUGE-L**
Binary	0.0795	0.0625	0.0523	0.0442	0.2427
Ternary	0.0830	0.0640	0.0526	0.0436	0.2564
Multiclass	0.0781	0.0616	0.0515	0.0435	0.2517

Table 3: Classification and Text Generation Results

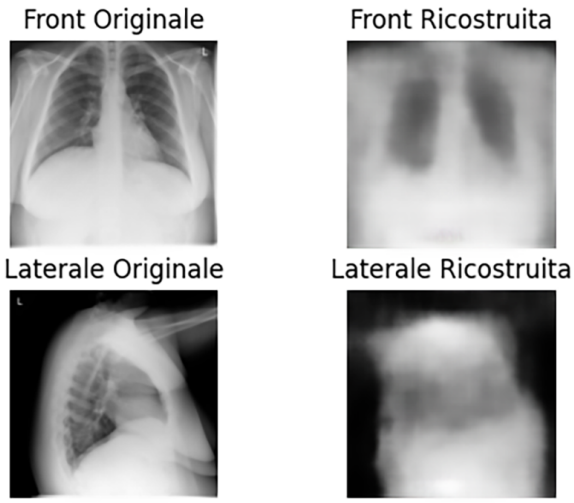

Figure 2: Examples of image reconstructions.

To address these issues, we modified our approach by separating the tasks into two distinct models:

1. **AllegroChirurgoModel** for classification and report generation.
2. **VAEModel** for image reconstruction.

aiming to isolate the challenges and better understand the sources of errors.

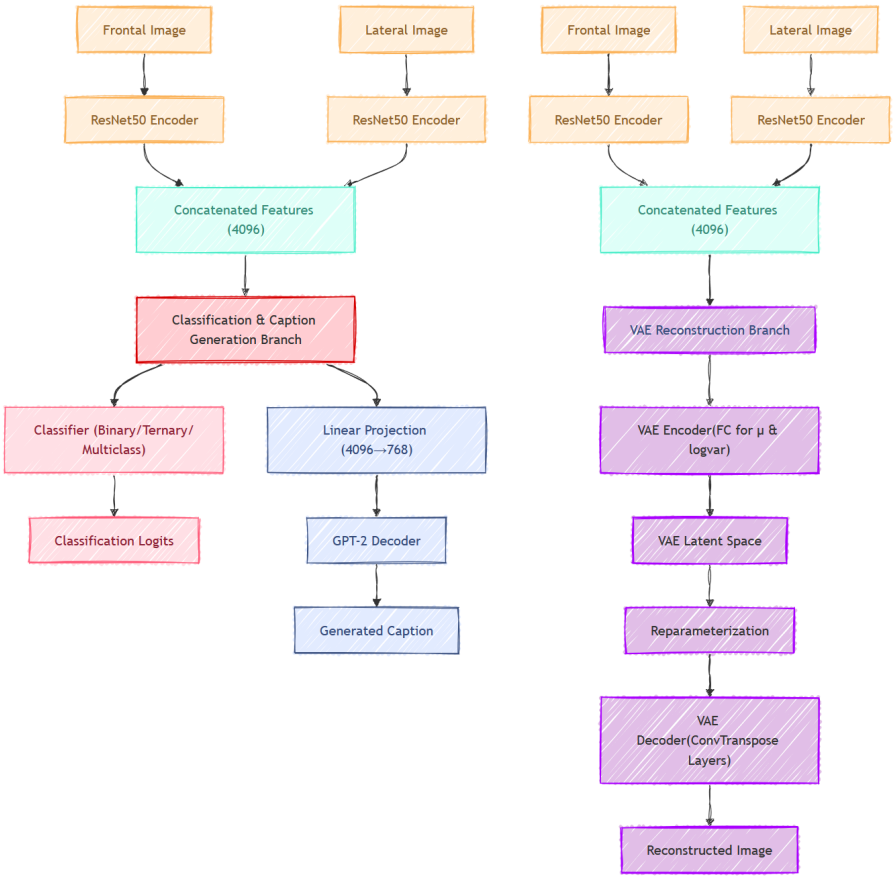

Figure 3: Architecture for the classification and caption generation model.

6.1 Results with the Separated Architecture

This new architecture allowed us to obtain better results, maintaining a similar (or slightly higher) classification accuracy—especially in multi-class scenarios, where the previous model's accuracy approached zero for almost all classes—and improving text generation by significantly boosting BLEU scores and reducing meaningless word generation.

Additionally, the VAE model showed enhanced image reconstruction quality. However, this objective remains less significant when combined with the necessity of unifying all three branches in a single model.

True Caption:
The cardiac contours are normal. The lungs are clear. Thoracic spondylosis.

Pred Caption (Model Output):
The cardiac silhouette is within normal limits for appearance in the contour of the frontal XXXX. No focal area of pulmonary consolidation is identified. No pneumothorax or effusion is identified.

	Class 1	**Class 2**	**Class 3**	**Class 4**	**Class 5**	**Total Accuracy**
Binary	74.63%	67.45%	-	-	-	70.73%
Ternary	49.27%	58.53%	37.90%	-	-	50.37%
Multiclass	70.73%	6.67%	54.32%	25.27 %	12.77%	41.94%

Table 4: Classification and Text Generation Results

	BLEU-1	**BLEU-2**	**BLEU-3**	**BLEU-4**	**ROUGE-L**
Binary	0.2649	0.2369	0.2207	0.2067	0.2375
Ternary	0.2900	0.2639	0.2485	0.2352	0.2410
Multiclass	0.2109	0.1865	0.1724	0.1606	0.2334

Table 5: Classification and Text Generation Results

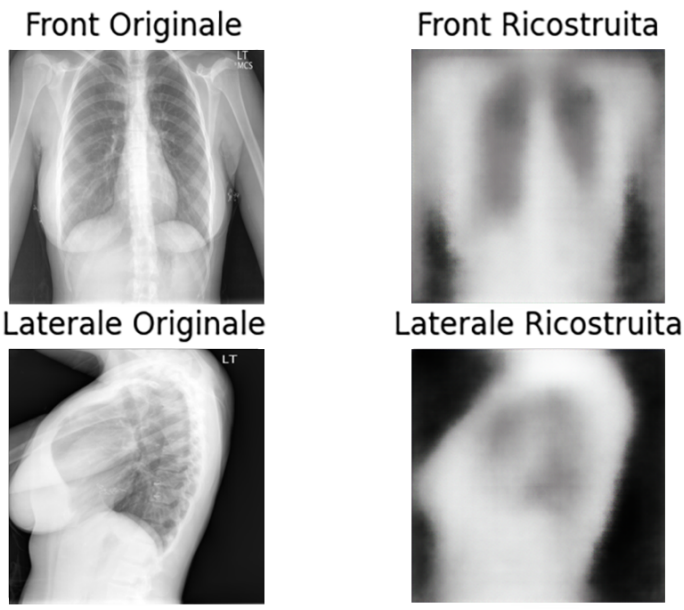

Figure 4: Examples of image reconstructions.

From these tables, it is evident that multi-class models—due to having many underrepresented classes—generally incur a more significant classification loss. This loss heavily impacts the latent space, leading to substantially worse results overall.

6.2 Architecture with the Custom Transformer

We also tried an architecture with a **Custom Transformer** that draws inspiration from the GPT-2 architecture but implements a lighter, more flexible design. This choice offers direct control over the model's structure and core components:

1. We apply a custom **Positional Encoding** to the *input embeddings* to provide the model with information about the position of tokens in the sequence. This is essential for the Transformer to keep track of word order and capture relationships between distant tokens.

2. **Transformer Encoder:** This module leverages the *self-attention* mechanism to weigh the importance of different tokens during prediction, while the *feed-forward network* further processes the information coming out of the self-attention layer. During training, we apply *sub-sequence masking* to prevent the model from "seeing" future tokens and to learn how to predict the next word based solely on the preceding context.

3. The **Forward Method** processes the input tokens in order by passing them through:

   - An embedding layer.
   - The positional encoding layer.
   - The *Transformer Encoder* block (which can be repeated multiple times, depending on the chosen depth).

   The output of the Transformer Encoder is then fed into a linear layer (`nn.Linear`) to predict the next token in the sequence.

4. During **Generation** the model predicts tokens one at a time, based on all previously generated tokens. Each predicted token is appended to the input sequence, and this process continues until an end-of-sequence token (e.g., `<EOS>`) is produced or a predefined maximum length is reached.

Our goal is to provide a more modular approach compared to GPT-2, while retaining the benefits of an attention mechanism that models long-range relationships. This design allowed us to experiment with various configurations of *layers*, *heads*, and embedding sizes, ultimately finding a good balance between generalization capability and computational efficiency.

True Caption:
The cardiac contours are normal. The lungs are clear. Thoracic spondylosis.

Pred Caption (Model Output):
cardiomediastinal silhouette and hilar contours. Lungs are clear without focal consolidation, pneumothorax, large pleural effusion, or pneumothorax. Cardiomediastinal contours are unremarkable. Negative for acute bony findings. Stable degenerative changes of the thoracic spine. No XXXX. Stable mild degenerative changes of the thoracic spine. Degenerative changes noted within the thoracic spine. Visualized skeletal structures are within the thoracic spine. Calcified right. Stable cardiaphragm. Stable normal limits.

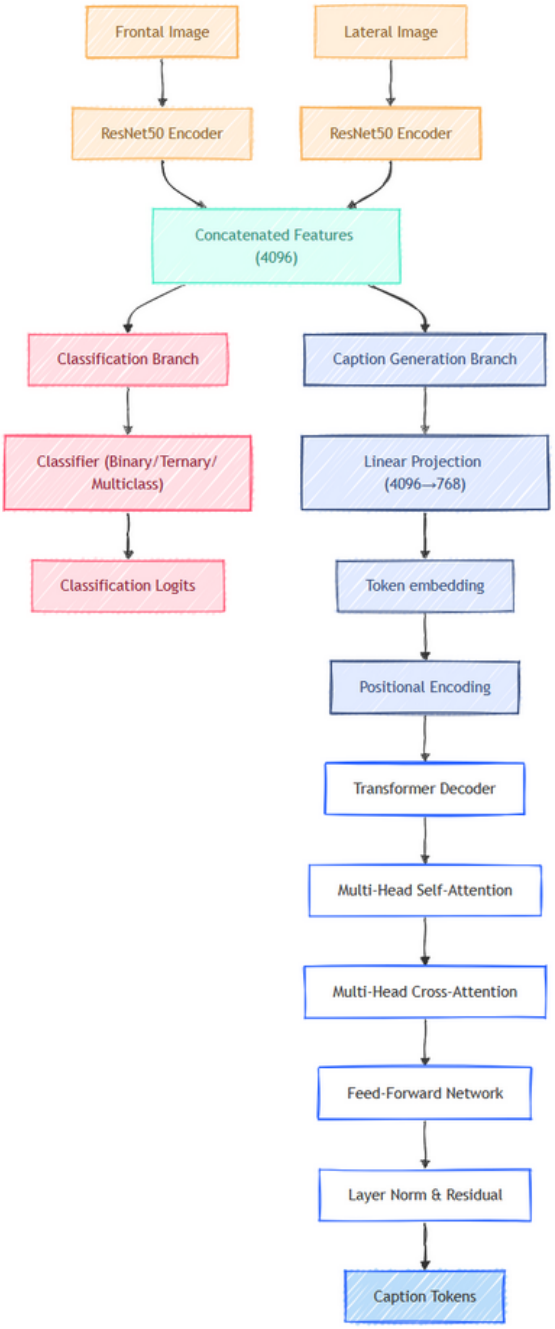

Figure 5: Architecture for the classification and caption generation model.

	Class 1	Class 2	Class 3	Class 4	Class 5	Total Accuracy
Binary	80.00%	61.29%	-	-	-	68.32%
Ternary	33.17%	61.75%	45.97%	-	-	47.44%
Multiclass	63.90%	17.74%	15.38%	8.51%	19.11%	34.62%

Table 6: Classification and Text Generation Results

	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-L
Binary	0.2074	0.1468	0.1093	0.0782	0.1875
Ternary	0.1981	0.1398	0.1045	0.0757	0.1767
Multiclass	0.2019	0.1407	0.1039	0.0735	0.1790

Table 7: Classification and Text Generation Results

7 Conclusions

In this work, we presented a comprehensive investigation of multi-task deep learning approaches for chest X-ray analysis, exploring various architectural configurations and their impact on performance. Our project yielded several key findings:

- The initial unified architecture, *while theoretically promising*, demonstrated limitations in practice. The simultaneous optimization of classification, report generation, and image reconstruction proved challenging, particularly given the constraints of our dataset size and the complexity of medical imaging tasks.
- The separation of tasks into specialized models yielded substantial improvements. The **classification accuracy** in binary scenarios increased from **65.75%** to **70.73%**, while **BLEU-4** scores for report generation showed a significant enhancement from **0.0442** to **0.2067**, indicating more coherent and relevant text generation.
- Our custom Transformer implementation, while showing promise in classification tasks (*68.32% accuracy in binary classification*), demonstrated mixed results in report generation, suggesting that pre-trained models like GPT-2 might be more suitable and economically/time efficient for this task.
- The impact of class imbalance remained a persistent challenge, particularly in multi-class scenarios where accuracy varied significantly across different categories (ranging from 8.51% to 63.90%).